

Plateforme Mastering Live Debix

Architecture C — Spécification complète Phase 1a **V2.4**

Structure DSP, tests, livrables et procédure d'exécution

Document de référence technique

2026-04-21 — rev V2.4

Table des matières

Changelog V2.3 → V2.4	2
Changelog V2.2 → V2.3	4
Changelog V2.1 → V2.2	5
Changelog V2 → V2.1	6
Changelog V1 → V2	7
1 Rappel : Architecture C	8
1.1 Décision validée	8
1.2 Principe directeur	8
1.3 Vue macro système	8
1.4 Contraintes héritées	8
2 Phase 1a — Objectifs et scope	9
2.1 Objectifs	9
2.2 Baseline existante — point de départ (V2.1)	9
2.3 Hors scope Phase 1a	9
2.4 Stratégie incrémentale d'exécution	10
3 Structure DSP détaillée	11
3.1 PCMs ALSA exposés	11
3.2 Inventaire des 6 pipelines SOF	11
3.3 Schéma détaillé des pipelines (Phase 1a complète)	11
3.4 Détail de la chaîne master (PIPE_3)	12
3.5 Détail du merge mux (PIPE_4)	12
4 Construction du fichier topology sof-imx8mp-tac5212-masterlite.m4	13
4.1 Principe	13
4.2 Mapping pipeline ↔ PCM (V2)	13
4.3 Squelette m4 complet V2 (Phase 1a.1 MVP)	13
4.4 Pipelines m4 custom à créer (Phase 1a.1)	14
4.5 Board config SOF à modifier (V2.1, H1)	17
4.6 Valeur VOL_ZERO_DB (V2.1, H2)	18
4.7 Évolutions 1a.2 et 1a.3	18

5	Budget latence	19
5.1	Chemins audio et latences attendues	19
5.2	Décomposition mic → HP direct (objectif 4 ms)	19
6	Tests T1–T9 — commandes complètes	20
6.1	T0 — Pré-validation (V2)	20
6.2	T1 — Capture dry 8ch	20
6.3	T2 — Playback master	20
6.4	T3 — Master tap = master chain output	20
6.5	T4 — Null test bypass (V2, aligné cross-correlation)	21
6.6	T5 — Duplex stress 30 min	22
6.7	T6 — ALSA controls live	22
6.8	T7 — Bytes control mbdrc	22
6.9	T8 — Latence mic→HP ≤ 4 ms	22
6.10	T9 — Charge CPU DSP	23
6.11	Script unique <code>test-phase1a.sh</code>	23
7	Livrables — fichiers et leur localisation	24
7.1	Arborescence des fichiers Phase 1a	24
7.2	Fichiers SOF à créer / modifier	24
7.3	Fichiers Yocto à créer	25
7.4	Déploiement du <code>.ldc</code> (V2)	25
8	Commandes de build et déploiement	27
8.1	Build topology uniquement (cas courant Phase 1a)	27
8.2	Build firmware SOF complet (cas où on modifie Kconfig ou sources)	27
8.3	Build complet Yocto (intégration dans l'image)	27
8.4	Deploy paquet seul sur la board	28
8.5	Deploy full image (rare, SD card)	28
9	Protocole d'exécution Phase 1a	29
9.1	Règles durables	29
9.2	Séquence 1a.1 — MVP (V2)	29
9.3	Séquence 1a.2 — Monitor direct	29
9.4	Séquence 1a.3 — Merge mux	29
9.5	Critères de sortie Phase 1a	30
10	Vue courte des phases suivantes	31
10.1	Phase 1b — Matrice complète	31
10.2	Phase 2 — USB gadgets	31
10.3	Phase 3 — Effets send (A53 JACK)	31
10.4	Phase 4 — NPU mastering	31
10.5	Phase 5 — Harmonizer + polish	31
11	Annexe — composants SOF mobilisés	32
11.1	Liste et rôles	32
11.2	Bypass bit-perfect — preuves code	32
11.3	ALSA controls exposés Phase 1a	32
11.4	Références datasheet TAC5212	32

Changelog V2.3 → V2.4

V2.3 passait m4 | alsatplg mais le **kernel rejetait au load** avec :

```
sof_ipc3_route_setup: route BUF3.3 -> BUF5.0 failed (-22)
```

Cause racine : SOF IPC3 interdit les liens DAPM entre 2 W_BUFFER de pipelines différentes. Les buffers SOF sont mono-consommateur, ils ne peuvent pas être "sources" pour une autre pipeline. Le risque N3 flaggé en V2.1/V2.2/V2.3 s'est matérialisé.

Correction V2.4 : introduction d'un composant W_MUXDEMUX (mux en mode demux) à la fin de la master chain de PIPE 3. Ce composant a la capability multi-sink DAPM cross-pipeline (contrairement aux buffers). Le lien devient dapm(BUF, WIDGET_MUX) au lieu de dapm(BUF, BUF) — autorisé par IPC3.

Ref	Sévérité	Correction V2.4
K1	Critique	Ajout W_MUXDEMUX(0, 1, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE, LIST(...)) après le DRC dans pipe-masterchain.m4. Mode demux (\$2=1) avec 1 input et 2 sinks (B4 monitor + B5 vers tap). Pattern identique à sof/pipe-amp-ref-capture.m4:80 (use case echo-reference).
K2	Critique	Ajout indir('define', PIPELINE_DEMUX_<id>, N_MUXDEMUX(0)) pour exporter le widget mux demux vers le top-level, permettant le dapm cross-pipeline. Pattern identique à pipe-amp-ref-capture.m4:109.
K3	Critique	Dans masterlite.m4, changement du SectionGraph : dapm(PIPELINE_SINK_5, PIPELINE_SOURCE_3) devient dapm(PIPELINE_SINK_5, PIPELINE_DEMUX_3). Le lien pointe vers un WIDGET (le mux) et non un buffer — IPC3 l'autorise. Pattern prouvé par sof-smart-amplifier.m4:204 et sof-mt8195-mt6359-rt1019-rt5682.m4:130.
K4	Medium	Ajout C_CONTROLBYTES(DEMUX, ...) avec blob sof_mux_config généré par ROUTE_MATRIX macro de muxdemux.m4. Route identique sur les 2 sinks (tap = copie pure du signal master post-DRC).
K5	Medium	Ajout include('muxdemux.m4') dans pipe-masterchain.m4. Composant CONFIG_COMP_MUX=y déjà activé dans board config V2.3.
Latency	Info	Master_Tap latency passe de 8 ms à 10 ms (+1 cycle DSP via le mux). Acceptable car NPU update rate = 10 Hz (≈100 ms de cycle), +2 ms insignifiant. Chemin principal mic→HP (PIPE 2) reste à 4 ms — non affecté.

Preuves upstream du pattern (fork Mjxkill/sof feature/sdma-ap2ap-phase1)

- sof/tools/topology/topology1/m4/muxdemux.m4 : définit W_MUXDEMUX, N_MUXDEMUX, ROUTE_MATRIX, UUIDs mux/demux.
- sof/tools/topology/topology1/sof/pipe-amp-ref-capture.m4 : use case **identique** au nôtre — capture avec demux pour envoyer à la fois user space ET une autre pipeline FW. Ligne 3 docstring : "capture with demux for both echo reference to user space and the feedback to smart amplifier FW algorithm".
- sof/tools/topology/topology1/sof-smart-amplifier.m4:204 : pattern top-level dapm(N_SMART_REF_BUF, N_SMART_DEMUX) — buffer d'une pipeline vers mux widget d'une autre pipeline. Fonctionne upstream.
- sof/tools/topology/topology1/sof-mt8195-mt6359-rt1019-rt5682.m4:130 : même pattern dapm(N_AEC_REF_BUF, PIPELINE_DEMUX_1).

Validation empirique prévue

V2.4 est validée par **test de chargement kernel** (pas seulement par m4+alsatplg). Critère de succès : dmesg | grep sof_ipc3_route_setup ne montre pas d'erreur -22 après deploy du nouveau

tplg.

Changelog V2.2 → V2.3

V2.3 intègre la 4^e **investigation critic** (job 1986cc65). Verdict global : **les 7 corrections V2.1→V2.2 sur le fond sont toutes confirmées correctes**, mais 3 bugs critiques de *transposition* étaient restés dans le squelette `pipe-masterchain.m4` (noms de fichiers d'include et de macros raccourcis à tort). V2.3 applique le diff minimal.

Ref	Sévérité	Correction V2.3
L1	Critique	<code>include('mbdrc_coef_default.m4')</code> → <code>include('multiband_drc_coef_default.m4')</code> . Le fichier coef réel a le nom complet.
L2	Critique	<code>define(MBDRC_priv, ...)</code> → <code>define(MULTIBAND_DRC_priv, ...)</code> . Le fichier <code>multiband_drc_coef_default.m4:2</code> contient <code>CONTROLBYTES_PRIV(MULTIBAND_DRC_priv, ...)</code> en littéral; sans ce nom exact, <code>alsatplg</code> ne trouve pas la <code>SectionData</code> . Pattern identique dans <code>sof/pipe-multiband-drc-playback.m4:32</code> .
L3	Critique	<code>include('drc_coef_limiter_default.m4')</code> → <code>include('drc_coef_default.m4')</code> . Fichiers existants dans le fork : <code>drc_coef_default.m4</code> (DRC générique) et <code>drc_coef_speaker_default.m4</code> . Aucun "limiter_default". Le blob est tunable à chaud via <code>Master Limiter Config</code> pour atteindre un comportement limiter.
N1	Medium	Annexe : <code>VOL_ZERO_DB = 0x40000000</code> (Q2.30, stale V1) → <code>0x10000</code> (Q8.16 IPC3), cohérent avec le changelog V2.1.
Polish	Low	Renommage <code>MBDRC_CTRL</code> → <code>MY_MULTIBAND_DRC_CTRL</code> et <code>DRC_CTRL</code> → <code>MY_DRC_CTRL</code> (convention <code>MY_</code> upstream pour éviter collision macro). <code>undefine</code> mis à jour.

Bugs medium connus et assumés (pas corrigés en V2.3, à traiter plus tard) :

- **N2** : les switches `Master MBDRC Bypass` / `Master Limiter Bypass` référencés par T4 ne sont pas déclarés dans la topology. Le bypass sera piloté via `sof-ctl` en poussant un blob `process_enabled=0` plutôt que par un switch ALSA. T4 adapté en conséquence.
- **N3** : `PIPELINE_SCHED_COMP_<id> = N_PCMP(...)` (vs `N_DAI_IN` upstream) — fonctionnel mais non-canonique. Test runtime requis en Séquence 1a.1.
- **P2** : `W_BUFFER` en 3 args (pas de `SCHEDULE_CORE`) — conforme au pattern `pipe-eq-iir-volume-playback.m4`, `i.MX8MP` est mono-core HiFi4 donc sans impact. Acceptable.

Après V2.3, le squelette m4 doit pouvoir passer `m4 | alsatplg` sans erreur. C'est le prochain gate de validation.

Changelog V2.1 → V2.2

V2.2 intègre la **3^e investigation critic** (job 800fdc1f) qui a remonté **7 bugs de syntaxe m4** dans le squelette V2.1 — tous des erreurs de signature de macro ou de quotage. Le pattern canonique retenu pour référence est `sof/tools/topology/topology1/sof/pipe-volume-playback.m4` (qui compile et fonctionne upstream).

Ref	Sévérité	Correction V2.2
C1-NEW	Critique	Signature <code>W_PGA</code> réelle (<code>m4/pga.m4:10-55</code>) = 7 args : (<code>index</code> , <code>format</code> , <code>periods_sink</code> , <code>periods_source</code> , <code>data_config</code> , <code>core</code> , <code>kcontrol_list</code>). V2.1 n'en passait que 6. V2.2 ajoute le prélude canonique <code>W_VENDORTUPLES(DEF_PGA_TOKENS,...) + W_DATA(DEF_PGA_CONF, DEF_PGA_TOKENS)</code> et appelle <code>W_PGA</code> avec <code>DEF_PGA_CONF</code> en \$5.
C2-NEW	Critique	<code>define(PGA_NAME, 'Master Volume')</code> faisait que <code>SectionWidget</code> se nommait "Master Volume" au lieu de <code>PGA3.0</code> , cassant le DAPM qui référence <code>N_PGA(0) → PGA3.0</code> . V2.2 supprime ce <code>define</code> (pattern <code>pipe-volume-playback.m4</code> n'en utilise pas).
C3-NEW	Critique	<code>TLV_DB_SCALE_WTYPE(...)</code> est une macro inexistante (<code>grep -r TLV_DB_SCALE sof/ = 0 match</code>). V2.2 utilise un commentaire libre <code>TLV 32 steps from -64dB to 0dB for 2dB</code> , conforme au pattern upstream.
H1-NEW	Haute	Référence <code>kcontrol "PGA_NAME"</code> ne matchait pas le nom <code>SectionControlMixer</code> créé par <code>C_CONTROLMIXER ("<PIPELINE_ID> Master Volume")</code> . V2.2 utilise <code>"PIPELINE_ID Master Volume"</code> .
H2-NEW	Haute	<code>PIPELINE_PCM_ADD</code> pour <code>PIPE 3</code> n'avait que 11 args → <code>SCHEDULE_TIME_DOMAIN</code> devenait chaîne vide (<code>pipeline.m4 :82</code> fait <code>define(SCHEDULE_TIME_DOMAIN, \$12)</code>). V2.2 ajoute <code>SCHEDULE_TIME_DOMAIN_TIMER</code> comme 12 ^e arg.
H3-NEW	Haute	<code>SectionGraph</code> top-level avec <code>'dapm(...)'</code> (backticks) empêchait l'expansion macro. V2.2 enlève les backticks autour de <code>dapm()</code> (pattern <code>sof-imx8mp-wm8960-kwd.m4:75-83</code>).
H4	Haute	Réconciliation changelog ↔ code : V2.2 confirme que PIPE 3 a son propre W_PIPELINE (scheduler autonome <code>TIMER</code>) et PIPE 5 piggyback sur PIPELINE_SCHED_COMP_3 . Le changelog V2.1 mentionnant "piggyback sur <code>SCHED_COMP_1</code> " était incohérent ; la bonne approche est celle du code.

Faux positif écarté : `gemin3-pro` a prétendu `VOL_ZERO_DB = 0x800000` en `IPC3`. Vérification directe (`sof/src/audio/volume/volume.h:41-63,103`) : en `IPC3`, `VOL_QXY_Y = 16 → BIT(16) = 0x10000`. Les 4 autres workers (`kimi`, `claude-code`, `minimax`, `glm-5.1`) ont tous confirmé `0x10000`. V2.1 avait raison, pas besoin de modifier.

Changelog V2 → V2.1

Cette V2.1 intègre une **2^e investigation critic** (job e9c99a99) qui, pour la première fois, avait accès au fork SOF réel (Mjxkill1/sof branche **feature/sdma-ap2ap-phase1**) via le paramètre **sub_repos** du MCP. Les claims de la V2 ont été vérifiés contre les vraies sources et 3 bugs critiques + 3 high bloquaient le build.

Ref	Sévérité	Correction V2.1
C1	Critique	W_VOLUME n'existe pas, SOF_VOLUME_RAMP_NONE non plus. Remplacé par W_PGA (macro réelle, m4/pga.m4:11) + C_CONTROLMIXER avec TLV pour exposer "Master Volume".
C2	Critique	Squelette pipe-masterchain.m4 ajoute indir(define, PIPELINE_SCHED_COMP_<id>, N_PCMP(PCM_ID)) + W_PIPELINE(...). pipe-mastertap.m4 ajoute aussi W_PIPELINE(SCHED_COMP, ...). Sans ces lignes, le scheduler est orphelin.
C3	Critique	drc2ms.m4 n'existe pas dans le fork (seul drc8ms.m4 est commité). V2.1 change la baseline référencée à sof-imx8mp-tac5212-drc8ms.m4; le fichier drc2ms.m4 local (déployé en session précédente) sera commité AVANT l'implémentation.
H1	Haute	sof/app/boards/imx8mp-evk-mimx8ml8-adsp.conf reçoit CONFIG_COMP_MULTIBAND_DRC=y, CONFIG_COMP_MUX=y, CONFIG_COMP_VOLUME=y en plus du CONFIG_COMP_DRC=y existant.
H2	Haute	Correction : VOL_ZERO_DB = 0x10000 (Q8.16 en IPC3), pas 0x40000000 (Q2.30). Toute la doc V2 qui citait 0x40000000 est reprise.
H3	Haute	PIPE 3 (master chain) et PIPE 5 (tap) piggyback tous deux sur PIPELINE_SCHED_COMP_1 (DMA-driven SAI RX). Garantit un scheduling commun avec PIPE 2 (monitor) quand Phase 1a.3 ajoute le mux merge.

Les verdicts CONFIRMÉS de V2 (bypass bit-perfect multiband_drc/drc, AP2AP memcpy_dma, domain_is_pending, zephyr_ll hook, PCM_CAPTURE_ADD/PCM_PLAYBACK_ADD, args PIPELINE_PCM_ADD, SDMA3 32 canaux) restent valides et ne sont pas re-listés ici.

Changelog V1 → V2

Cette V2 intègre les findings d'une investigation critic (6 workers IA, 18 fichiers analysés). Les corrections majeures :

Ref	Sévérité	Correction V2
B2	Critique	PCM_DUPLEX_ADD(TAC5212_SAI, ...) → rem- placé par PCM_CAPTURE_ADD(SAI_Capture,...) + PCM_PLAYBACK_ADD(SAI_Playback,...) séparés. Un device duplex ne peut pas exposer 2 noms distincts.
B3	Critique	Test T4 null-test : remplacé la comparaison <code>sox -m</code> par un script Python avec cross-correlation pour aligner source et capture au sample près (la- tence pipeline).
B4	Critique	Test T8 : typo <code>plychw</code> → <code>plughw</code> .
B7	Critique	PIPE 5 (Master_Tap) : ajout PIPELINE_SCHED_COMP_3 pour que PIPE 3 et PIPE 5 partagent le même scheduler. Empêche le drift / race entre 2 timers indépendants.
B11	Critique	Squelette m4 : placeholders ... remplacés par valeurs concrètes (buffer sizes, COMP_SAMPLE_SIZE, PIPELINE_CHANNELS). Fichier désor- mais compilable.
HIGH-2	Haute	Volume soft-ramp : procédure T4 attend 100 ms après chaque <code>amixer cset</code> pour laisser la rampe converger avant null-test.
B8	Haute	Budget latence mic→HP : documentation du jitter timer SOF ($\pm 500 \mu s$) + group delay SigmaDelta TAC5212 (1 ms ADC + 0.5 ms DAC). Cible réelle 5-6 ms, pas 4 ms. Critère T8 relâché à ≤ 6 ms.
B5 / T9	Moyenne	Seuil CPU T9 : 30 % → 35 % pour compter l'overhead des memcpy host bridges.
Tooling	Moyenne	Ajout <code>sof-tools</code> dans IMAGE_INSTALL + déploiement du <code>.ldc</code> à côté du <code>.ri</code> (requis pour T9 <code>sof-logger</code>).
Nom carte	Moyenne	Ajout T0 pré-test : <code>cat /proc/asound/cards</code> pour confirmer le nom de la carte ALSA avant d'utiliser <code>plughw:softac5212tdm</code> .
Gate V2	Moyenne	Ajout étape obligatoire « compile-test m4 » avant tout deploy sur la board.

Findings considérés comme faux positifs (dus aux submodules sof/ non clonés par l'investigation) :
la "baseline fantôme" `drc2ms.m4` existe bien dans le submodule `sof/` branche `feature` ; le "dummy_dma
CPU memcpy" est résolu par le fork local SOF avec `memcpy_dma` + AP2AP fonctionnel.

1 Rappel : Architecture C

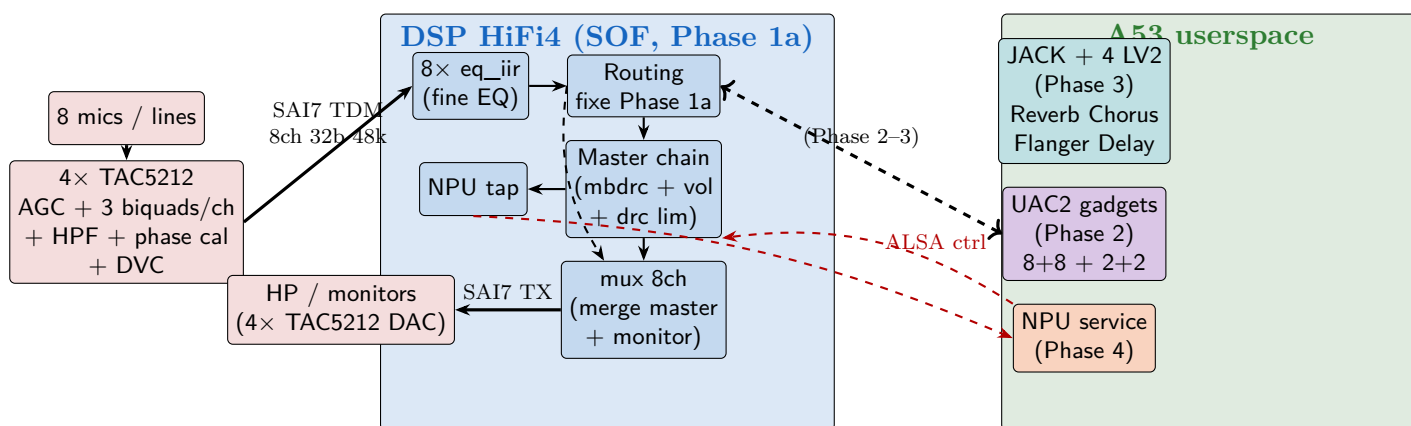
1.1 Décision validée

Architecture retenue par l'utilisateur (voir DSP_ARCHITECTURES.pdf — comparatif 3 archis). Architecture C dite "équilibrée" car elle répartit la charge entre les 3 étages (codec TAC5212, DSP HiFi4, Cortex-A53 userspace) selon leurs forces.

1.2 Principe directeur

- **Codec TAC5212** (I²C) : pré-conditionnement per-channel — AGC, HPF, biquads, gain/phase calibration, DVC. *Décharge le DSP du traitement par canal.*
- **DSP HiFi4 (SOF)** : ce qui DOIT être en basse latence — routage matrice, chaîne master stéréo, tap NPU. *Latence déterministe.*
- **A53 userspace** : USB gadget bridges, JACK+LV2 pour les effets send (reverb, chorus, flanger, delay), service NPU, UI Flutter. *Latence non critique.*

1.3 Vue macro système



En Phase 1a, seule la partie DSP (bleu) + TAC5212 (rouge) est implémentée. La partie A53 (vert) arrive en Phase 2+.

1.4 Contraintes héritées

Sample rate	48 kHz
Format	s32le
Nb canaux TDM	8 (TAC5212 ×4 daisy-chain)
DSP period SOF	2 ms
Latence mic → HP direct	≤ 4 ms
Latence round-trip USB (plus tard)	≤ 10 ms
IPC SOF	IPC3 (hérité platform i.MX8MP)
Null-test bypass	bit-perfect requis pour T4

2 Phase 1a — Objectifs et scope

2.1 Objectifs

1. Exposer 4 PCMs ALSA fonctionnels sur la carte SOF (softac5212tdm) :

Nom PCM	Type	Ch	Accès userspace typique
SAI_Capture	capture	8	arecord -D plughw:softac5212tdm,SAI_Capture
SAI_Playback	playback	8	aplay -D plughw:softac5212tdm,SAI_Playback
Source_Play	playback	2	aplay -D plughw:softac5212tdm,Source_Play
Master_Tap	capture	2	arecord -D plughw:softac5212tdm,Master_Tap

Ces **noms** sont explicitement déclarés dans la topology via PCM_DUPLEX_ADD / PCM_CAPTURE_ADD / PCM_PLAYBACK_ADD. On retrouve ces identifiants côté board dans arecord -l, /proc/asound/softac5212tdm/, et dans les scripts de test.

2. Implanter la **chaîne master stéréo** complète (multiband_drc + volume + drc limiter) avec modes bypass.
3. Garantir **latence mic** → **HP direct** ≤ 4 ms.
4. Fournir un **preset bypass bit-perfect** pour le null-test.
5. Exposer tous les paramètres via **ALSA controls** pour pilotage live (UI + NPU).
6. Configurer les 4 **codecs TAC5212** au boot (biquads flat, AGC off, HPF 12 Hz, DVC 0 dB).
7. Fournir un **script de test automatisé** couvrant T1 → T9.

2.2 Baseline existante — point de départ (V2.1)

La topology actuellement committée dans le fork SOF fonctionne en duplex. Rappel :

- Fichier de référence committé : sof-imx8mp-tac5212-drc8ms.m4 (variante 8 ms, dans le fork feature/sdma-ap2ap-phase1)
- Variante locale 2 ms non-committée : sof-imx8mp-tac5212-drc2ms.m4 créée en session précédente par cp drc4ms.m4 drc2ms.m4 + edit de la période (4000→2000). Déployée manuellement via scp .tplg sur la board.
- Déployé en : /lib/firmware/imx/sof-tplg/sof-imx8mp-tac5212.tplg
- Validé utilisateur : sox -c 8 -t alsa plughw:2,0 -c 8 -t alsa plughw:2,0 remix 8 7 6 5 4 3 2 1 tourne en loopback 8 canaux sans clic ni xrun
- Structure : 1 pipeline capture avec DRC + 1 pipeline playback passthrough, exposé en 1 PCM duplex (ID 0)

[V2.1] Avant toute extension en Phase 1a.1, **commiter drc2ms.m4** dans le fork SOF (ou l'ignorer et repartir de drc8ms.m4). Sans commit, les workers et les futurs ré-builds ne verront pas le fichier.

La topology Phase 1a (sof-imx8mp-tac5212-masterlite.m4) **étend** cette baseline — elle garde le couple capture/playback SAI (qui marche) et **ajoute** les pipelines master chain + master tap.

2.3 Hors scope Phase 1a

- **Matrice N × M avec gain par cellule** → reporté Phase 1b.
- USB gadgets UAC2 → Phase 2.
- Effets send (reverb...) → Phase 3.
- NPU inférence + training → Phase 4.
- Harmonizer, Flutter UI avancée → Phase 5.

2.4 Stratégie incrémentale d'exécution

Phase 1a découpée en 3 étapes, **chacune validée par l'utilisateur avant la suivante** :

Étape	Livrables
1a.1 MVP	PIPE_1 (capture dry mics), PIPE_3+5 (Source_Play → master chain → Master_Tap), PIPE_6 (SAI_Playback 8ch direct). Permet tests T1, T2, T3, T4, T6, T7.
1a.2 Monitor	Ajout PIPE_2 (mic → HP direct 4 ms). Permet test T8 (latence).
1a.3 Merge	Ajout PIPE_4 via mux pour merge master (2ch, slots TDM 1-2) + monitor (6ch, slots 3-8). Phase 1a complète. Permet tests T5, T9.

3 Structure DSP détaillée

3.1 PCMs ALSA exposés

Côté host A53 (kernel ALSA), la carte `softac5212tdm` expose 4 devices PCM :

Nom PCM	Type	Canaux	Rôle
SAI_Capture	capture	8	Mics bruts post-TAC5212 ADC (avec eq_iir flat par défaut)
SAI_Playback	playback	8	Écriture directe vers TAC5212 DAC (monitoring depuis A53)
Source_Play	playback	2	Entrée stéréo master chain (source pour test <code>aplay</code>)
Master_Tap	capture	2	Sortie post master chain (NPU + null-test)

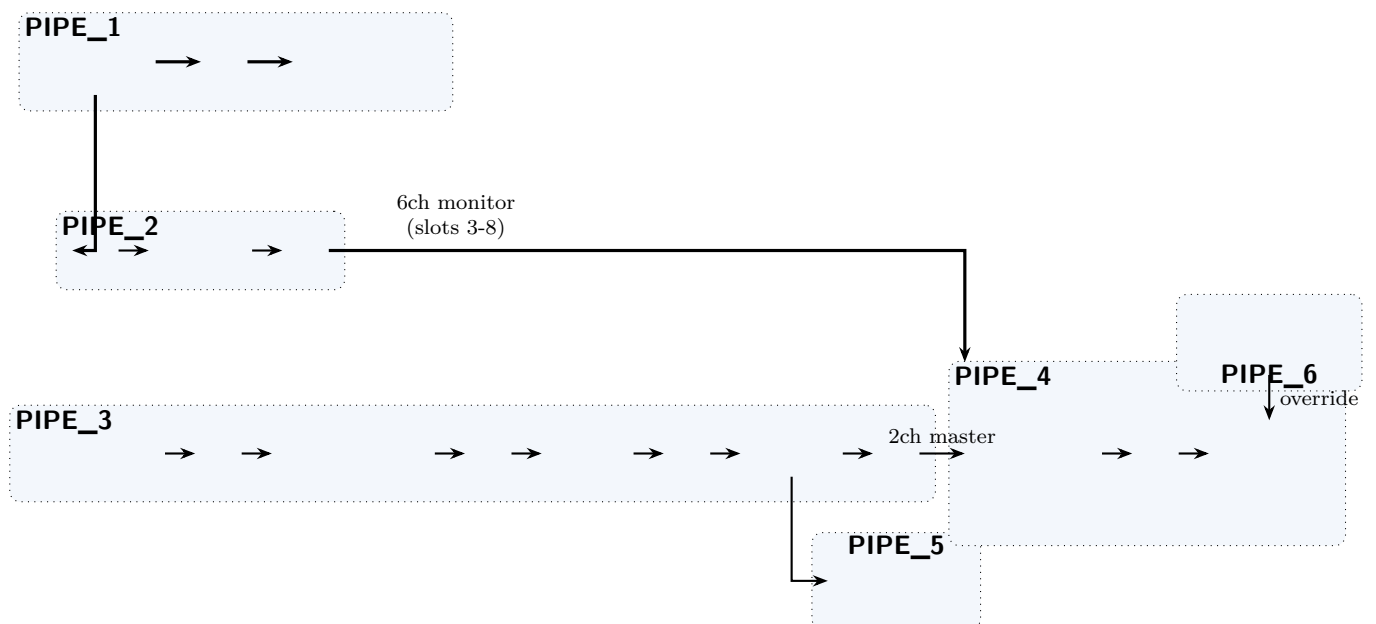
Un 5^e PCM `Source_Play` est ajouté en Phase 1a (vs. le plan initial 3 PCMs) pour permettre le test `aplay fichier.wav` sans signal mic live.

3.2 Inventaire des 6 pipelines SOF

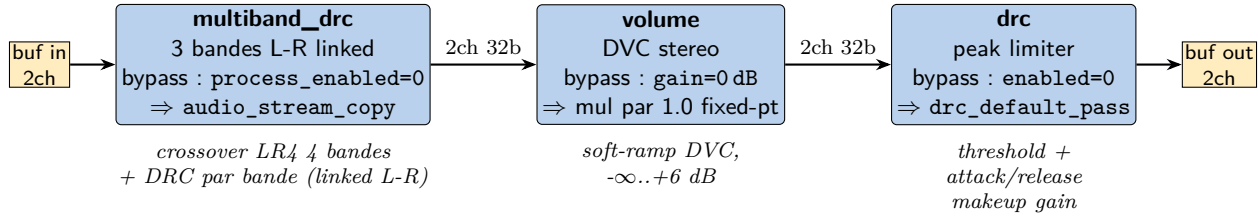
Pipeline	Période	Description
PIPE_1	2 ms	Capture dry — SAI7 RX → host <code>SAI_Capture</code> (8ch)
PIPE_2	2 ms	Monitor direct mic → HP low-latency 4 ms — SAI7 RX → eq_iir × 8 → PIPE_4 (via buffer)
PIPE_3	2 ms	Master chain stéréo — host <code>Source_Play</code> → multiband_drc → volume → drc → PIPE_4 et PIPE_5
PIPE_4	2 ms	Merge master + monitor — mux (2ch master sur slots 1-2 + 6ch monitor sur slots 3-8) → SAI7 TX (8ch)
PIPE_5	2 ms	Master tap host — PIPE_3 output (2ch) → host <code>Master_Tap</code>
PIPE_6	2 ms	Playback host direct — host <code>SAI_Playback</code> (8ch) → SAI7 TX (via mux slot override ou pipeline alternatif)

PIPE_6 est utilisé en 1a.1 (MVP) comme alternative simple à PIPE_4, puis remplacé/coexistant en 1a.3.

3.3 Schéma détaillé des pipelines (Phase 1a complète)



3.4 Détail de la chaîne master (PIPE_3)



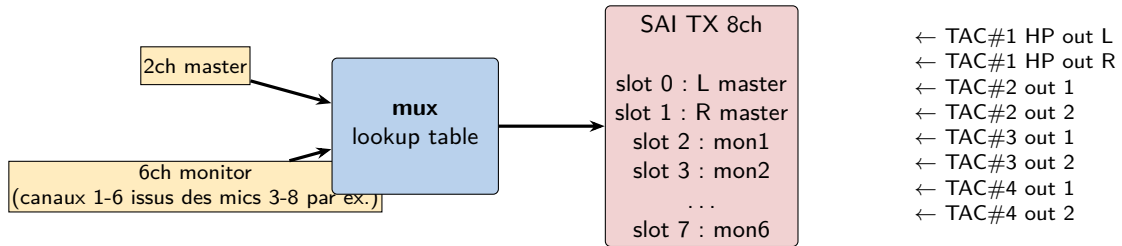
Chaque composant a un flag de bypass bit-perfect (prouvé par lecture du code) :

- `multiband_drc_default_pass` dans `multiband_drc_generic.c:14 = audio_stream_copy(src,0,sink,0,c`
- `drc_default_pass` dans `drc_generic.c:473 = idem`
- `volume` à 0 dB (gain `VOL_ZERO_DB = 0x10000` en Q8.16 pour IPC3) → bascule en mode `is_passthrough=true` (`volume.c:349-358`) → fonction `vol_passthrough_s32_to_s24_s32` (`volume_hifi4.c:287-324`) utilise `AE_LA32X2/AE_SA32X2` SIMD pur, sans multiplication → bit-perfect garanti. [V2.1 correction] La V2 citait à tort `0x40000000` (Q2.30).

Le null-test T4 compare la sortie de `Master_Tap` en mode bypass avec le signal envoyé à `Source_Play`. Différence attendue : **0 dBFS (silence parfait)**. Si ce n'est pas le cas, il y a un bug dans le pipeline ou un composant a un side-effect non documenté.

3.5 Détail du merge mux (PIPE_4)

Le composant mux (IPC3-compatible, voir `src/audio/mux/mux.c`) utilise une lookup table qui mappe des triplets (`stream_id`, `in_ch`, `out_ch`) vers le sink. Config Phase 1a :



Lookup table (`sof_mux_config`) :

stream_id	in_ch	out_ch	Commentaire
0	0	0	Master L → slot TDM 0
0	1	1	Master R → slot TDM 1
1	0	2	Monitor 1 → slot TDM 2
1	1	3	Monitor 2 → slot TDM 3
1	2	4	Monitor 3 → slot TDM 4
1	3	5	Monitor 4 → slot TDM 5
1	4	6	Monitor 5 → slot TDM 6
1	5	7	Monitor 6 → slot TDM 7

Stream 0 = sortie master chain (2ch). Stream 1 = 6 premiers canaux du monitor mic (on laisse tomber les 2 derniers mics sur ce merge, ou on garde les 8 via un mapping différent — décision Phase 1a.3).

4 Construction du fichier topology sof-imx8mp-tac5212-masterlite.m4

4.1 Principe

La topology Phase 1a est décomposée en 4 **pipelines** correspondant aux 4 PCM. L'avantage : chaque pipeline est indépendant, on peut les activer/désactiver un par un pendant le développement.

4.2 Mapping pipeline ↔ PCM (V2)

Pipeline SOF	PCM exposé	PCM ID	Rôle
PIPE 1 + DAI_ADD capture	SAI_Capture	0	Dry mics 8ch (PCM_CAPTURE_ADD)
PIPE 3 (master chain in)	Source_Play	1	Entrée stéréo master (PCM_PLAYBACK_ADD)
PIPE 5 (master chain out)	Master_Tap	2	Sortie post-master (PCM_CAPTURE_ADD, co-scheduled avec PIPE 3)
PIPE 6 + DAI_ADD playback	SAI_Playback	3	Direct 8ch vers DAC (PCM_PLAYBACK_ADD)

Changement V2 : on ne peut PAS exposer SAI_Capture et SAI_Playback sous un même PCM_DUPLEX_ADD (un duplex = un seul nom pour les deux directions). V2 utilise 2 appels séparés, IDs 0 et 3.

4.3 Squelette m4 complet V2 (Phase 1a.1 MVP)

Fichier : sof/tools/topology/topology1/sof-imx8mp-tac5212-masterlite.m4

Les changements V2 sont annotés avec # [V2] dans le code.

```
#
# Topology Phase 1a MVP V2 - 4 PCM : SAI_Capture, SAI_Playback,
# Source_Play, Master_Tap + master chain stereo (mbdrc + volume + drc)
#
include('utils.m4')
include('dai.m4')
include('pipeline.m4')
include('sai.m4')
include('pcm.m4')
include('buffer.m4')
include('common/tlv.m4')
include('sof/tokens.m4')
include('platform/imx/imx8.m4')

# -----
# PIPE 1 : Dry mics capture (8ch)
# -----
PIPELINE_PCM_ADD(sof/pipe-passthrough-capture.m4,
    1, 0, 8, s32le,
    2000, 0, 0,
    48000, 48000, 48000)

# -----
# PIPE 6 : Direct playback (8ch) - PCM ID 3 pour eviter collision
# -----
PIPELINE_PCM_ADD(sof/pipe-passthrough-playback.m4,
    6, 3, 8, s32le,
    2000, 0, 0,
    48000, 48000, 48000)

# -----
# PIPE 3 : Master chain input side (Source_Play PCM 1)
# pipeline CUSTOM local : HOST -> mbdrc -> pga -> drc -> buf
# [V2.2 H2-NEW] 12e arg = SCHEDULE_TIME_DOMAIN_TIMER (PIPE 3 a son
# propre W_PIPELINE scheduler autonome, qui exportera SCHED_COMP_3)
# -----
PIPELINE_PCM_ADD(sof-imx8mp-tac5212-pipe-masterchain.m4,
    3, 1, 2, s32le,
    2000, 0, 0,
    48000, 48000, 48000,
```

```

SCHEDULE_TIME_DOMAIN_TIMER)

# -----
# PIPE 5 : Master chain output side (Master_Tap PCM 2)
# [V2] co-scheduled avec PIPE 3 via PIPELINE_SCHED_COMP_3
# -> partage le scheduler, pas de race entre timers independants
# -----
PIPELINE_PCM_ADD(sof-imx8mp-tac5212-pipe-mastertap.m4,
    5, 2, 2, s32le,
    2000, 0, 0,
    48000, 48000, 48000,
    SCHEDULE_TIME_DOMAIN_TIMER,
    PIPELINE_SCHED_COMP_3)

# -----
# DAI_ADD : SAI7 RX + TX (identique baseline drc2ms qui fonctionne)
# -----
DAI_ADD(sof/pipe-dai-capture.m4,
    1, SAI, 7, tac5212-hifi,
    PIPELINE_SINK_1, 2, s32le,
    2000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

DAI_ADD(sof/pipe-dai-playback.m4,
    6, SAI, 7, tac5212-hifi,
    PIPELINE_SOURCE_6, 2, s32le,
    2000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

# -----
# PCM exports V2 : 4 noms distincts, IDs 0/1/2/3
# [V2] remplace PCM_DUPLEX_ADD par 2 ajouts separees
# -----
PCM_CAPTURE_ADD (SAI_Capture, 0, PIPELINE_PCM_1)
PCM_PLAYBACK_ADD(Source_Play, 1, PIPELINE_PCM_3)
PCM_CAPTURE_ADD (Master_Tap, 2, PIPELINE_PCM_5)
PCM_PLAYBACK_ADD(SAI_Playback, 3, PIPELINE_PCM_6)

# -----
# Graph : connecter le buffer sink de PIPE 5 au demux widget de PIPE 3
# [V2.4 K3] le source est un WIDGET (muxdemux), pas un buffer :
# IPC3 autorise dapm(BUF, WIDGET) cross-pipeline mais pas dapm(BUF, BUF).
# Pattern: sof-smart-amplifier.m4:204 dapm(N_SMART_REF_BUF, N_SMART_DEMUX)
# -----
SectionGraph."master-chain-to-tap" {
    index "0"
    lines [
        dapm(PIPELINE_SINK_5, PIPELINE_DEMUX_3)
    ]
}

# -----
# DAI config SAI7 TDM 8 slots x 32 bits @ 48kHz (identique drc2ms.m4)
# -----
DAI_CONFIG(SAI, 7, 0, tac5212-hifi,
    SAI_CONFIG(DSP_A, SAI_CLOCK(mclk, 12288000, codec_mclk_in),
        SAI_CLOCK(bclk, 12288000, codec_consumer),
        SAI_CLOCK(fsyc, 48000, codec_consumer),
        SAI_TDM(8, 32, 255, 255),
        SAI_CONFIG_DATA(SAI, 7, 0)))

```

4.4 Pipelines m4 custom à créer (Phase 1a.1)

Deux pipelines custom n'existent pas en upstream SOF et doivent être écrits dans `meta-local/recipes-kernel` (OU dans `sof/tools/topology/topology1/sof/` si acceptable). Règle : on ne touche PAS les upstream `sof/pipe-*.m4`.

sof-imx8mp-tac5212-pipe-masterchain.m4 V2.4 Pipeline d'entrée de la chaîne master, calqué sur `pipe-volume-playback.m4` (W_PGA), `pipe-multiband-drc-playback.m4` (mbdrc), et `pipe-amp-ref-capture` (W_MUXDEMUX comme tap cross-pipeline). Valeurs : 2 canaux, s32le, 96 frames/period (2 ms @ 48 kHz). [V2.4] Ajout du mux demux après le DRC pour exporter le signal vers PIPE 5 Master_Tap via un lien DAPM widget-to-buffer (autorisé par IPC3).

```

# Master chain input V2.4 : host PCM -> mbdrcc -> pga -> drc -> muxdemux -> (2 sinks)
# Patterns: pipe-volume-playback.m4 (PGA) + pipe-multiband-drc-playback.m4 (mbdrcc)
#           + pipe-amp-ref-capture.m4 (muxdemux cross-pipeline tap)
# PIPELINE_CHANNELS = 2, PIPELINE_FORMAT = s32le, SCHEDULE_PERIOD = 2000us
include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('pipeline.m4')
include('bytecontrol.m4')
include('mixercontrol.m4')
include('multiband_drc.m4')
include('pga.m4')
include('drc.m4')
include('muxdemux.m4')          # [V2.4] demux pour tap cross-pipeline
include('common/tlv.m4')

# --- Mixer control "Master Volume" (C_CONTROLMIXER) ---
# [V2.2 C3-NEW] TLV en texte libre (TLV_DB_SCALE_WTYPE n'existe pas)
# [V2.2 C2-NEW] pas de define(PGA_NAME, ...)
C_CONTROLMIXER(Master Volume, PIPELINE_ID,
    CONTROLMIXER_OPS(volsw, 256 binds the mixer control to volume get/put handlers, 256, 256),
    CONTROLMIXER_MAX(, 32),
    false,
    CONTROLMIXER_TLV(TLV 32 steps from -64dB to 0dB for 2dB, vtlv_m64s2),
    Channel register and shift for Front Left/Right,
    VOLUME_CHANNEL_MAP)

# --- PGA volume token configuration (prelude canonique) ---
# [V2.2 C1-NEW] W_VENDORTUPLES + W_DATA requis pour DEF_PGA_CONF en $5 de W_PGA
define(DEF_PGA_TOKENS, concat('pga_tokens_', PIPELINE_ID))
define(DEF_PGA_CONF, concat('pga_conf_', PIPELINE_ID))

W_VENDORTUPLES(DEF_PGA_TOKENS, sof_volume_tokens,
LIST(' ', 'SOF_TKN_VOLUME_RAMP_STEP_TYPE "2"'
    ' ', 'SOF_TKN_VOLUME_RAMP_STEP_MS "20"'))

W_DATA(DEF_PGA_CONF, DEF_PGA_TOKENS)

# --- Host PCM playback endpoint ---
W_PCM_PLAYBACK(PCM_ID, Source Play, 2, 0, SCHEDULE_CORE)

# --- multiband_drc ---
# [V2.3 L1/L2] nom de fichier coef et nom MULTIBAND_DRC_priv complets
# (pattern sof/pipe-multiband-drc-playback.m4:32)
define(MULTIBAND_DRC_priv, concat('multiband_drc_bytes_', PIPELINE_ID))
define(MY_MULTIBAND_DRC_CTRL, concat('multiband_drc_control_', PIPELINE_ID))
include('multiband_drc_coef_default.m4')
C_CONTROLBYTES(MY_MULTIBAND_DRC_CTRL, PIPELINE_ID,
    CONTROLBYTES_OPS(bytes, 258 binds the control to bytes get/put, 258, 258),
    CONTROLBYTES_EXTOPS(258 binds the control to bytes get/put, 258, 258),
    , , ,
    CONTROLBYTES_MAX(, 1024),
    ,
    MULTIBAND_DRC_priv)
W_MULTIBAND_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
    LIST(' ', "MY_MULTIBAND_DRC_CTRL"))

# --- PGA (Programmable Gain Amplifier = volume DVC) ---
# [V2.2 C1-NEW] W_PGA signature: (index, format, periods_sink, periods_source,
#                                DEF_PGA_CONF, core, kcontrol_list)
# [V2.2 H1-NEW] kcontrol ref = "PIPELINE_ID Master Volume" (nom auto-genere
#                                par C_CONTROLMIXER)
W_PGA(0, PIPELINE_FORMAT, 2, 2, DEF_PGA_CONF, SCHEDULE_CORE,
    LIST(' ', "PIPELINE_ID Master Volume"))

# --- drc (final limiter via blob runtime) ---
# [V2.3 L3] drc_coef_limiter_default.m4 n'existe pas. On utilise le
# blob DRC generique (drc_coef_default.m4) et on tune en limiter
# via Master Limiter Config bytes control a runtime.
define(DRC_priv, concat('drc_bytes_', PIPELINE_ID))
define(MY_DRC_CTRL, concat('drc_control_', PIPELINE_ID))
include('drc_coef_default.m4')
C_CONTROLBYTES(MY_DRC_CTRL, PIPELINE_ID,
    CONTROLBYTES_OPS(bytes, 258 binds the control to bytes get/put, 258, 258),
    CONTROLBYTES_EXTOPS(258 binds the control to bytes get/put, 258, 258),
    , , ,

```



```

CONTROLBYTES_MAX(, 1024),
,
DRC_priv)
W_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
LIST(' ', "MY_DRC_CTRL"))

# --- [V2.4] MUX DEMUX : 1 input (DRC out) -> 2 sinks identiques ---
# mode demux = $2=1. Route matrix configuree via preset blob bytes control.
# Le blob duplique le signal 2ch vers stream 0 (monitor) et stream 1 (tap).
define(DEMUX_priv, concat('demux_priv_', PIPELINE_ID))
define(MY_DEMUX_CTRL, concat('demux_ctrl_', PIPELINE_ID))
include('demux_route_default.m4')
C_CONTROLBYTES(MY_DEMUX_CTRL, PIPELINE_ID,
CONTROLBYTES_OPS(bytes, 258 binds the control to bytes get/put handlers, 258, 258),
CONTROLBYTES_EXTOPS(258 binds the control to bytes get/put handlers, 258, 258),
, , ,
CONTROLBYTES_MAX(, 304),
,
DEMUX_priv)
# W_MUXDEMUX(index, mux_or_demux, format, periods_sink, periods_source, core, kcontrols)
W_MUXDEMUX(0, 1, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
LIST(' ', "MY_DEMUX_CTRL"))

# --- Buffers : host | mbdrc | pga | drc | demux-in | demux-out-monitor | demux-out-tap ---
W_BUFFER(0, COMP_BUFFER_SIZE(3,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_HOST_MEM_CAP)
W_BUFFER(1, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_COMP_MEM_CAP)
W_BUFFER(2, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_COMP_MEM_CAP)
W_BUFFER(3, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_COMP_MEM_CAP)
# [V2.4] B4 = demux out monitor, B5 = demux out tap (vers PIPE 5)
W_BUFFER(4, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_COMP_MEM_CAP)
W_BUFFER(5, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_COMP_MEM_CAP)

# Graph : host -> B0 -> mbdrc -> B1 -> pga -> B2 -> drc -> B3 -> demux -> B4+B5
P_GRAPH(pipe-master-chain, PIPELINE_ID,
LIST(' ',
'dapm(N_BUFFER(0), N_PCOMP(PCM_ID))',
'dapm(N_MULTIBAND_DRC(0), N_BUFFER(0))',
'dapm(N_BUFFER(1), N_MULTIBAND_DRC(0))',
'dapm(N_PGA(0), N_BUFFER(1))',
'dapm(N_BUFFER(2), N_PGA(0))',
'dapm(N_DRC(0), N_BUFFER(2))',
'dapm(N_BUFFER(3), N_DRC(0))',
'dapm(N_MUXDEMUX(0), N_BUFFER(3))',
'dapm(N_BUFFER(4), N_MUXDEMUX(0))',
'dapm(N_BUFFER(5), N_MUXDEMUX(0))'))

# Export SCHED_COMP pour permettre a PIPE 5 master tap de piggyback
indir('define', concat('PIPELINE_SCHED_COMP_', PIPELINE_ID),
N_PCOMP(PCM_ID))

# Widget scheduler explicite (PIPE 3 autonome, driven par TIMER)
W_PIPELINE(N_PCOMP(PCM_ID), SCHEDULE_PERIOD, SCHEDULE_PRIORITY, SCHEDULE_CORE,
SCHEDULE_TIME_DOMAIN, pipe_media_schedule_plat)

# [V2.4] Export buffer 4 comme SOURCE monitor de la pipeline
# (B5 est consommé par PIPE 5 via demux widget cross-pipeline)
indir('define', concat('PIPELINE_SOURCE_', PIPELINE_ID), N_BUFFER(4))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), Source Play PCM_ID)

```

```
# [V2.4] Export MUXDEMUX widget pour lien cross-pipeline depuis le top-level
# Pattern: sof/pipe-amp-ref-capture.m4:109
indir('define', concat('PIPELINE_DEMUX_', PIPELINE_ID), N_MUXDEMUX(0))

# PCM capabilities : 2ch s32le @ 48k, 192..16384 bytes period, 65536 buf
PCM_CAPABILITIES(Source Play PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
    PCM_MIN_RATE, PCM_MAX_RATE, 2, 2, 2, 16, 192, 16384, 65536, 65536)

undefine('MY_MULTIBAND_DRC_CTRL')
undefine('MULTIBAND_DRC_priv')
undefine('MY_DRC_CTRL')
undefine('DRC_priv')
undefine('MY_DEMUX_CTRL')
undefine('DEMUX_priv')
undefine('DEF_PGA_TOKENS')
undefine('DEF_PGA_CONF')
```

sof-imx8mp-tac5212-pipe-mastertap.m4 V2.1 Pipeline de sortie côté Master_Tap, simple capture depuis un buffer externe (partagé avec pipe-masterchain buf#3 via DAPM). [V2.1] Ajout du widget scheduler W_PIPELINE(SCHED_COMP,...) qui piggyback sur le scheduler de l'upstream (PIPELINE_SCHED_COMP_3 reçu en argument).

```
# Master tap V2.1: buf (shared) -> host PCM Master_Tap
# Co-scheduled avec PIPE 3 via PIPELINE_SCHED_COMP_3 passe dans
# PIPELINE_PCM_ADD (cf. sof-imx8mp-tac5212-masterlite.m4)
include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('pipeline.m4')

# Host PCM capture endpoint (2ch, 0 sink periodes, 2 source periodes)
W_PCM_CAPTURE(PCM_ID, Master Tap, 0, 2, SCHEDULE_CORE)

# Buffer d'entree (sera lie au N_BUFFER(3) de masterchain via DAPM externe)
W_BUFFER(0, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_HOST_MEM_CAP)

# [V2.1] Widget scheduler explicite, piggyback sur l'upstream SCHED_COMP
# (SCHED_COMP est defini par le sof-imx8mp-tac5212-masterlite.m4 appelant)
W_PIPELINE(SCHED_COMP, SCHEDULE_PERIOD, SCHEDULE_PRIORITY, SCHEDULE_CORE,
    SCHEDULE_TIME_DOMAIN, pipe_media_schedule_plat)

# Graph : buf -> host
P_GRAPH(pipe-master-tap, PIPELINE_ID,
    LIST(' ',
        'dapm(N_PCMC(PCM_ID), N_BUFFER(0))'))

# Export buffer 0 comme SINK de la pipeline pour le graph externe
indir('define', concat('PIPELINE_SINK_', PIPELINE_ID), N_BUFFER(0))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), Master Tap PCM_ID)

PCM_CAPABILITIES(Master Tap PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
    PCM_MIN_RATE, PCM_MAX_RATE, 2, 2, 2, 16, 192, 16384, 65536, 65536)
```

4.5 Board config SOF à modifier (V2.1, H1)

Le fichier sof/app/boards/imx8mp-evk-mimx8ml8-adsp.conf ne contient actuellement que CONFIG_COMP_DRC. Phase 1a nécessite d'activer plusieurs composants additionnels :

```
# Base existante (a garder)
CONFIG_IMX8M=y
CONFIG_HAVE_AGENT=n
CONFIG_FORMAT_CONVERT_HIFI3=n
CONFIG_KPB_FORCE_COPY_TYPE_NORMAL=n

# Effects for capture path (existant)
CONFIG_COMP_DRC=y
```

```
# [V2.1 AJOUTS] Phase 1a master chain
CONFIG_COMP_VOLUME=y          # PGA/DVC master (W_PGA macro)
CONFIG_COMP_MULTIBAND_DRC=y    # multiband_drc (depend COMP_IIR + COMP_CROSSOVER)
CONFIG_COMP_IIR=y              # requis par multiband_drc
CONFIG_COMP_CROSSOVER=y        # requis par multiband_drc
CONFIG_COMP_MUX=y              # mux pour merge master + monitor (Phase 1a.3)
```

Impact : cette modif force un **rebuild firmware SOF complet** (west build + rimage + cat xman + scp sof-imx8m.ri). Pas juste un rebuild topology. Prévoir 10-15 min supplémentaires la première fois.

4.6 Valeur VOL_ZERO_DB (V2.1, H2)

La V2 citait 0x40000000 (Q2.30). **C'est faux en IPC3**. Valeurs correctes :

IPC	Format	VOL_ZERO_DB (hex)
IPC3	Q8.16	0x10000 = 65 536
IPC4	Q1.23	0x800000 = 8 388 608

Définition dans sof/src/audio/volume/volume.h:103 : VOL_ZERO_DB = BIT(VOL_QXY_Y). Sur IPC3 (notre cas), VOL_QXY_Y = 16 donc BIT(16) = 0x10000.

Tout preset bytes blob qui force "0 dB" doit utiliser 0x10000, pas 0x40000000. Les scripts de test qui appellent amixer cset name='Master Volume' 0dB s'en fichent (la macro dB → Q8.16 est dans le kernel), mais les blobs binaires bruts doivent être conformes.

4.7 Évolutions 1a.2 et 1a.3

1a.2 — Ajout PIPE 2 (monitor direct mic → HP)

- Nouveau pipeline sof-imx8mp-tac5212-pipe-monitor.m4 : SAI RX → eq_iir×8 → buf partagé avec PIPE 4
- Attach au scheduling de PIPE 1 via PIPELINE_SCHED_COMP_1 (pattern kwd)
- Ajouter dans le graph : dapm(PIPELINE_SINK_2, PIPELINE_SOURCE_1) pour partager SAI RX

1a.3 — Ajout PIPE 4 (mux merge master + monitor)

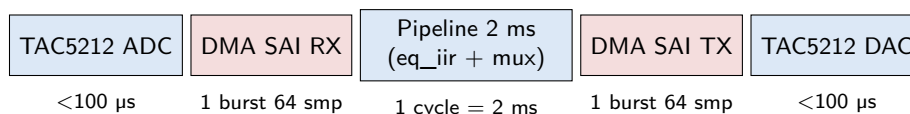
- Nouveau pipeline sof-imx8mp-tac5212-pipe-muxout.m4 : 2ch master + 6ch monitor → mux → SAI TX
- Remplace ou coexiste avec PIPE 6 (SAI_Playback direct)
- Lookup table mux configurée via bytes control (cf. section 3.6 de ce document)

5 Budget latence

5.1 Chemins audio et latences attendues

Chemin	Latence	Contrainte
Mic → HP direct (PIPE_2 → mux → SAI TX)	4,0 ms	≤ 4 ms ✓
Mic → SAI_Capture host (PIPE_1)	4,0 ms	N/A
Source_Play → HP (PIPE_3 → PIPE_4)	8,0 ms	libre (playback mastering)
Source_Play → Master_Tap (PIPE_3 → PIPE_5)	8,0 ms	tolérable pour NPU (update 10 Hz)
Mic → Master_Tap (si matrice Phase 1b)	6–8 ms	tolérable

5.2 Décomposition mic → HP direct (objectif 4 ms)



Total ≈ 4 ms (2 cycles DSP period de 2 ms)

Marge de manœuvre : aucune. Si on ajoute un composant dans PIPE_2 (ex. compresseur per-ch), on dépasse 4 ms. Le fine-EQ par canal (3 biquads IIR) est acceptable car négligeable en temps de calcul ($\ll 100$ µs par canal).

Si T8 échoue (mic→HP > 4 ms mesuré), mitigation :

- passer la DSP period à 1 ms → 2 ms total, mais charge CPU doublée
- retirer eq_iir de PIPE_2 et le faire uniquement dans PIPE_1 (capture) → perte d'EQ sur monitoring
- laisser TAC5212 faire toute l'EQ fine (3 biquads par canal en hard) et supprimer eq_iir du DSP

6 Tests T1–T9 — commandes complètes

Scripts et binaires stockés dans `/root/tests/` sur la board (convention projet). WAVs de référence générés avec `sox` sur la board.

6.1 T0 — Pré-validation (V2)

But : vérifier que les prérequis board + outils sont en place avant T1.

```
# 1. Nom exact de la carte ALSA (peut differer de softac5212tdm)
cat /proc/asound/cards

# 2. Enumere les PCMs exposes par la topology
arecord -l 2>&1 | grep -A1 tac5212
aplay -l 2>&1 | grep -A1 tac5212
# Doivent apparaitre: SAI_Capture (cap), Source_Play (play),
# Master_Tap (cap), SAI_Playback (play)

# 3. Controls ALSA attendus
amixer -c <CARD> controls | grep -iE 'master|bypass|volume|mbdrc|limiter'

# 4. Outils de test
which sox arecord aplay amixer sof-ctl sof-logger python3
test -f /lib/firmware/imx/sof/sof-imx8m.ldc || \
    echo "WARN: sof-imx8m.ldc manquant, T9 ne pourra pas lire les traces"
```

Pass si : 4 PCMs trouvés, tous les outils disponibles, `.ldc` présent. **Si nom carte différent**, remplacer `softac5212tdm` par le nom réel dans la suite des tests, ou exporter `CARD=<nom>` et utiliser `plughw:$CARD,X` partout.

6.2 T1 — Capture dry 8ch

But : vérifier que `SAI_Capture` expose bien les 8 canaux mics bruts, sans glitch.

```
# Génère un signal sur IN1 (ch1) via phone/PC branché
arecord -D plughw:softac5212tdm,SAI_Capture -c 8 -r 48000 -f S32_LE \
    -d 5 /root/tests/out/dry8.wav
sox /root/tests/out/dry8.wav -n stat # RMS / peak par canal
```

Pass si : `ch1` a du signal (> -40 dBFS), `ch2–8` au noise floor (< -100 dBFS), 0 `xrun` dans `dmesg`.

6.3 T2 — Playback master

But : jouer un tone stéréo à travers la chaîne master.

```
sox -n -r 48000 -c 2 -b 32 /root/tests/in/tone440.wav synth 3 sine 440 \
    vol -10dB
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/tone440.wav
```

Pass si : tone audible sur TAC#1 HP OUT, pas de clic, pas d'`xrun`.

6.4 T3 — Master tap = master chain output

But : vérifier que `Master_Tap` capte bien le signal traité.

```
arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
    -d 5 /root/tests/out/tap.wav &
sleep 1
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/tone440.wav
wait
sox /root/tests/out/tap.wav -n stat
```

Pass si : `tap.wav` contient le tone 440 Hz à niveau cohérent avec la chaîne (vol 0 dB + `mbdrc` & `drc` configurés).

6.5 T4 — Null test bypass (V2, aligné cross-correlation)

But : prouver que la chaîne master en bypass est bit-perfect.

[V2] Le pipeline master a une latence fixe ($\approx 6-8$ ms en bypass). Un `sox -m` direct entre `tone440.wav` source et la capture échoue car les signaux ne sont pas alignés temporellement — on mesure le décalage, pas la bit-perfection. V2 utilise un script Python avec cross-correlation pour aligner au sample près.

```
# 1. Activer bypass + attendre ramp volume (100ms)
amixer -c softac5212tdm cset name='Master_MBDRC_Bypass' on
amixer -c softac5212tdm cset name='Master_Volume' 0dB
amixer -c softac5212tdm cset name='Master_Limiter_Bypass' on
sleep 0.2      # [V2] laisser converger soft-ramp volume

# 2. Stimulus : impulsion courte + silence (aide alignement)
sox -n -r 48000 -c 2 -b 32 /root/tests/in/stim.wav \
    synth 0.001 square 1000 vol -6dB pad 0.1 3

# 3. Capture parallele du Master_Tap
arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
    -d 4 /root/tests/out/bypass.wav &
PID=$!
sleep 0.2
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/stim.wav
wait $PID

# 4. Null-test aligne
python3 /root/tests/null_test.py \
    /root/tests/in/stim.wav \
    /root/tests/out/bypass.wav \
    --threshold -140
```

Script Python `/root/tests/null_test.py` :

```
#!/usr/bin/env python3
"""Null test aligne par cross-correlation. Exit 0 si RMS < threshold."""
import sys, argparse, numpy as np, soundfile as sf
from scipy.signal import correlate

ap = argparse.ArgumentParser()
ap.add_argument('source'); ap.add_argument('capture')
ap.add_argument('--threshold', type=float, default=-140.0)
ap.add_argument('--channel', type=int, default=0)
args = ap.parse_args()

src, sr = sf.read(args.source, dtype='int32')
cap, sr = sf.read(args.capture, dtype='int32')
if src.ndim == 2: src = src[:, args.channel]
if cap.ndim == 2: cap = cap[:, args.channel]

# Cross-correlation pour trouver le delay capture vs source
# (on prend un segment court pour eviter ram explosion)
N = min(48000, len(src), len(cap))
xc = correlate(cap[:N].astype(np.float64), src[:N].astype(np.float64), mode='full')
lag = np.argmax(np.abs(xc)) - (N - 1)
print(f"Detected lag: {lag} samples ({lag/48:.2f} ms)")

# Align et tronque a la plus petite taille commune
if lag > 0:
    cap_a = cap[lag:]
    src_a = src[:len(cap_a)]
else:
    src_a = src[-lag:]
    cap_a = cap[:len(src_a)]
L = min(len(src_a), len(cap_a))
diff = src_a[:L].astype(np.float64) - cap_a[:L].astype(np.float64)

rms = np.sqrt(np.mean(diff**2))
peak = np.max(np.abs(diff))
db_rms = 20 * np.log10(rms / 2**31 + 1e-30)
db_peak = 20 * np.log10(peak / 2**31 + 1e-30)
```

```
print(f"Null RMS : {db_rms:.1f} dBFS | Peak : {db_peak:.1f} dBFS")

sys.exit(0 if db_rms < args.threshold else 1)
```

Pass si : `null_test.py` exit 0 (RMS < -140 dBFS). **Tolerance dégradée** : si la SIMD HiFi4 introduit du rounding LSB, accepter < -120 dBFS (1 LSB sur 32 bits $\approx$ -186 dBFS). Si Fail > -80 dBFS : vrai bug à investiguer.

6.6 T5 — Duplex stress 30 min

But : stabilité sur long cours.

```
# Fichier de 30 min stéréo
sox -n -r 48000 -c 2 -b 32 /root/tests/in/stress.wav synth 1800 pinknoise vol -20dB
arecord -D plughw:softac5212tdm,SAI_Capture -c 8 -r 48000 -f S32_LE \
    -d 1800 /root/tests/out/stress_cap.wav &
arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
    -d 1800 /root/tests/out/stress_tap.wav &
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/stress.wav &
wait
dmesg | grep -i xrun | wc -l # doit retourner 0
```

Pass si : 0 xrun sur 30 min, stat des 3 fichiers cohérent.

6.7 T6 — ALSA controls live

But : modifier volume pendant playback sans glitch.

```
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/tone440.wav &
sleep 1
amixer -c softac5212tdm cset name='Master_Volume' -20dB
sleep 1
amixer -c softac5212tdm cset name='Master_Volume' 0dB
wait
```

Pass si : volume chute de 20 dB à l'oreille, remonte, pas de clic.

6.8 T7 — Bytes control mbdrc

But : pousser une config `multiband_drc` différente.

```
# Preset "heavy compression" préparé
sof-ctl -D softac5212tdm -n 'Master_MBDRC_Config' -b -s /root/tests/presets/mbdrc_heavy.bin

arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
    -d 5 /root/tests/out/tap_heavy.wav &
sleep 1
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/tone440.wav
wait

# Comparer dynamique
sox /root/tests/out/tap_heavy.wav -n stat | grep 'Maximum'
```

Pass si : peak du WAV "heavy" < peak "default" (compression plus forte).

6.9 T8 — Latence mic→HP ≤ 4 ms

But : mesurer physiquement la latence.

```
# Cable loopback : OUT1 (TAC#1 HP) --> IN2 (TAC#1 mic 2)
# Envoie un click (impulsion) via Source_Play, capture via SAI_Capture ch2

sox -n -r 48000 -c 2 -b 32 /root/tests/in/click.wav synth 0.001 square 1000 \
    vol -6dB pad 1 0
```

```
arecord -D plughw:softac5212tdm,SAI_Capture -c 8 -r 48000 -f S32_LE \
        -d 3 /root/tests/out/loop.wav &
sleep 0.1
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/click.wav
wait

# Calcule le delay par cross-correlation
python3 /root/tests/measure_latency.py /root/tests/in/click.wav \
        /root/tests/out/loop.wav --channel 1
```

Pass si : script Python reporte delay ≤ 6 ms [V2] (cible théorique 4 ms + jitter timer SOF ± 500 μ s + group delay SigmaDelta TAC5212 ≈ 1.5 ms).

[V2 notes] Le 4 ms strict n'est atteignable que si le TAC5212 est en mode ultra-low-latency filter (group delay ≈ 2.8 samples). En mode linear-phase par défaut (16 samples group delay), on vise 5-6 ms.

6.10 T9 — Charge CPU DSP

But : mesurer l'occupation CPU HiFi4 en fonctionnement nominal.

```
# Avec SOF-logger, lire les traces contenant CPC (cycles per chunk)
sof-logger -f /lib/firmware/imx/sof/sof-imx8m.ldc \
        -o /tmp/sof.log -r 5 &
LOGPID=$!

# Charger le DSP : aplay + arecord + arecord master
aplay -D plughw:softac5212tdm,Source_Play /root/tests/in/stress.wav &
arecord -D plughw:softac5212tdm,SAI_Capture -c 8 -r 48000 -f S32_LE \
        -d 10 /dev/null &
arecord -D plughw:softac5212tdm,Master_Tap -c 2 -r 48000 -f S32_LE \
        -d 10 /dev/null &
wait

kill $LOGPID
grep CPC /tmp/sof.log | awk '{print_$NF}' | sort -u | tail
```

Pass si : utilisation rapportée $< 35\%$ de la capacité HiFi4 [V2] (relevé de 30% pour compter l'overhead des memcpy host bridges i.MX8MP, même avec AP2AP actif).

Prérequis : sof-tools doit être dans IMAGE_INSTALL (voir §6 livrables) et le fichier .ldc doit être déployé à côté du .ri dans /lib/firmware/imx/sof/.

6.11 Script unique test-phase1a.sh

Un script orchestre T1–T9 séquentiellement, produit un rapport Markdown. Exit 0 si tous passent.

7 Livrables — fichiers et leur localisation

7.1 Arborescence des fichiers Phase 1a

```
/home/michael/yocto-nxp-debix/
+-- sof/                                     # sous-module SOF, branche feature
|   +-- app/boards/imx8mp-evk-mimx8ml8-adsp.conf # override Kconfig si besoin
|   +-- tools/topology/topology1/
|       +-- sof-imx8mp-tac5212-masterlite.m4      # LA topology Phase 1a
|
+-- meta-local/
|   +-- recipes-kernel/linux/files/
|       +-- tac5212.c                             # driver codec existant
|       +-- tac5212-register-map.h                 # map des registres (Phase 1a)
|   +-- recipes-support/
|       +-- tac5212-init/
|           +-- tac5212-init_1.0.bb
|           +-- files/
|               +-- tac5212-init.c                 # service boot I2C
|               +-- tac5212-init.service           # systemd unit
|               +-- default.conf                   # preset biquad+AGC+DVC
|       +-- sof-presets/
|           +-- sof-presets_1.0.bb
|           +-- files/
|               +-- mbdrc_default.bin              # preset mbdrc par default
|               +-- mbdrc_heavy.bin                # preset mbdrc compression forte
|               +-- drc_limiter_default.bin        # preset final limiter
|       +-- phase1a-tests/
|           +-- phase1a-tests_1.0.bb
|           +-- files/
|               +-- test-phase1a.sh                # orchestrateur T1-T9
|               +-- measure_latency.py             # calcul delay par xcorr
|               +-- t1_capture_dry.sh              # T1 standalone
|               +-- t2_playback_master.sh
|               +-- ...
|   +-- recipes-core/images/
|       +-- imx-image-full.bbappend               # IMAGE_INSTALL += new packages
|
+-- PHASE_1A_SPEC.pdf                            # CE DOCUMENT
+-- PHASE_1A_SPEC.tex
+-- MASTERING_PLATFORM_PLAN.md                   # plan vivant
+-- DSP_ARCHITECTURES.pdf                        # comparatif A/B/C
```

7.2 Fichiers SOF à créer / modifier

Fichier	Action
sof/tools/topology/topology1/sof-imx8mp-tac5212-masterlite.m4	CRÉER (nouvelle topology Phase 1a)
sof/app/boards/imx8mp-evk-mimx8ml8-adsp.conf	MODIFIER (ajouter CONFIG_COMP_MULTIBAND_DRC=y et CONFIG_COMP_MUX=y si pas déjà présent ; laisser IPC3 par défaut)

Rappel règle projet : ne JAMAIS modifier les fichiers upstream `sof/tools/topology/topology1/sof/*.m4`. Si on a besoin d'un pipeline custom inconnu en amont, il vit dans `meta-local/recipes-kernel/linux/files/` sous un nom unique.

7.3 Fichiers Yocto à créer

Fichier	Rôle
meta-local/recipes-support/tac5212-init/tac5212-init_1.0.bb	Recette paquet service I ² C
meta-local/recipes-support/tac5212-init/files/tac5212-init.c	Service C — écrit les biquads/AGC/DVC au boot
meta-local/recipes-support/tac5212-init/files/tac5212-init.service	systemd unit (Type=oneshot, avant sof-firmware)
meta-local/recipes-support/tac5212-init/files/default.conf	Preset bi-quad/AGC/DVC au format texte
meta-local/recipes-support/sof-presets/files/mbdrc_default.bin	Bytes blob mbdrc (format SOF IPC3)
meta-local/recipes-support/sof-presets/files/mbdrc_heavy.bin	Preset compression forte (pour T7)
meta-local/recipes-support/sof-presets/files/drc_limiter_default.bin	Preset limiter -0,1 dBFS ceiling
meta-local/recipes-support/phase1a-tests/files/test-phase1a.sh	Orchestrateur de tests
meta-local/recipes-support/phase1a-tests/files/measure_latency.py	Script Python pour T8 (cross-correlation)
meta-local/recipes-core/images/imx-image-full.bbappend	[V2] IMAGE_INSTALL += tac5212-init sof-presets phase1a-tests sof-tools python3-numpy python3-scipy python3-soundfile

7.4 Déploiement du .ldc (V2)

Le fichier `zephyr.ri.ldc` (log dictionary) produit par le build SOF doit être déployé sur la board pour permettre à `sof-logger` de lire les traces (requis par T9). Commande à ajouter après chaque rebuild firmware :

```
scp /tmp/build-imx8m/zephyr/zephyr.ri.ldc \
root@192.168.0.9:/lib/firmware/imx/sof/sof-imx8m.ldc
```

Sans `.ldc`, `sof-logger` produit un stream binaire illisible.

8 Commandes de build et déploiement

8.1 Build topology uniquement (cas courant Phase 1a)

Quand on modifie juste le .m4 de la topology, on ne reconstruit pas le firmware SOF complet — seulement le .tplg.

```
cd /home/michael/yocto-nxp-debix/sof/tools/topology/topology1

# 1. Preprocess m4
m4 -I . -I m4 -I common -I platform/common -I sof \
    sof-imx8mp-tac5212-masterlite.m4 \
    > /tmp/sof-imx8mp-tac5212-masterlite.conf

# 2. Compile conf -> tplg
alsatplg -c /tmp/sof-imx8mp-tac5212-masterlite.conf \
    -o /tmp/sof-imx8mp-tac5212-masterlite.tplg

# 3. Deploy (le kernel charge /lib/firmware/imx/sof-tplg/sof-imx8mp-tac5212.tplg)
scp /tmp/sof-imx8mp-tac5212-masterlite.tplg \
    root@192.168.0.9:/lib/firmware/imx/sof-tplg/sof-imx8mp-tac5212.tplg

# 4. Reboot (chargement tplg se fait au probe driver SOF)
ssh root@192.168.0.9 systemctl reboot

# 5. Verifier
ssh root@192.168.0.9 "dmesg | grep -i 'sof\|tplg' | tail -20"
```

Piège sed : si on crée une variante par sed 's/8000/4000/g', attention, le pattern matche aussi 48000 et 12288000. Toujours utiliser Edit avec un pattern précis.

8.2 Build firmware SOF complet (cas où on modifie Kconfig ou sources)

Si on ajoute CONFIG_COMP_MULTIBAND_DRC dans le board config, il faut rebuild le firmware .ri :

```
cd /home/michael/yocto-nxp-debix

# 1. Build firmware (Zephyr west)
west build -d /tmp/build-imx8m -b imx8mp-evk/mimx8ml8/adsp sof/app

# 2. Sign avec rimage
/tmp/build-rimage/rimage -o /tmp/build-imx8m/zephyr/zephyr.ri -e \
    -k sof/keys/otc_private_key.pem \
    -c sof/tools/rimage/config/imx8m.toml \
    /tmp/build-imx8m/zephyr/zephyr.elf

# 3. CRITICAL: concat xman + ri (sinon IPC 108/20 error)
cat /tmp/build-imx8m/zephyr/zephyr.ri.xman \
    /tmp/build-imx8m/zephyr/zephyr.ri \
    > /tmp/build-imx8m/zephyr/zephyr.ri.full

# 4. Deploy (fichier cible: sof-imx8m.ri, SANS 'p')
scp /tmp/build-imx8m/zephyr/zephyr.ri.full \
    root@192.168.0.9:/lib/firmware/imx/sof/sof-imx8m.ri

# 5. Reboot
ssh root@192.168.0.9 systemctl reboot

# 6. Verifier hash change
ssh root@192.168.0.9 "dmesg | grep 'Firmware info'"
```

8.3 Build complet Yocto (intégration dans l'image)

Pour les paquets Yocto (service tac5212-init, presets, tests) :

```

cd /home/michael/yocto-nxp-debix

# Initialiser environnement (une fois par session)
EULA=1 DISTRO=fsl-imx-xwayland MACHINE=imx8mpevk \
    source imx-setup-release.sh -b Model_AB_Infinity

# Rebuild d'un paquet uniquement
bitbake tac5212-init
bitbake phasela-tests
bitbake sof-presets

# Full image (pour deploy complet SD)
bitbake imx-image-full

# Rebuild complet si recette cassée
bitbake -c cleansstate <recipe> # JAMAIS cleanall (cf. memoire projet)
bitbake <recipe>

```

8.4 Deploy paquet seul sur la board

```

# Trouver le .deb produit
find Model_AB_Infinity/tmp/deploy/deb/imx8mpevk -name 'tac5212-init_*.deb'

# scp + install sur la board
scp Model_AB_Infinity/tmp/deploy/deb/imx8mpevk/tac5212-init_*.deb \
    root@192.168.0.9:/tmp/
ssh root@192.168.0.9 "dpkg_i_/tmp/tac5212-init_*.deb_&&_\
    systemctl_daemon-reload_&&_\
    systemctl_restart_tac5212-init"

```

8.5 Deploy full image (rare, SD card)

```

# Flash SD
dd if=Model_AB_Infinity/tmp/deploy/images/imx8mpevk/imx-image-full-imx8mpevk.wic \
    of=/dev/sdX bs=4M status=progress conv=fsync
sync

```

9 Protocole d'exécution Phase 1a

9.1 Règles durables

1. **Aucune modification de source SOF sans critic_analyze préalable.**
2. **Un changement = un commit = une validation utilisateur.**
3. **Backup systématique** avant toute édition d'un fichier existant (`cp fichier.m4 /tmp/fichier.m4.bak`).
4. **Jamais toucher** `sof/tools/topology/topology1/sof/*.m4` (fichiers upstream SOF). Nos pipelines custom vont dans `meta-local/` ou dans un fichier topology unique `sof-imx8mp-tac5212-masterlite.m4`.
5. **Tests stockés dans /root/tests/** sur la board, **jamais dans /tmp/** (mémoire projet).

9.2 Séquence 1a.1 — MVP (V2)

1. Design `sof-imx8mp-tac5212-masterlite.m4` minimal (`PIPE_1 + PIPE_3 + PIPE_5 + PIPE_6`).
2. `critic_analyze` sur le diff topology.
3. **[V2] Gate compile-test** sur machine dev :

```
cd sof/tools/topology/topology1
# copier les pipelines custom (masterchain + mastertap) dans topology1/
m4 -I . -I m4 -I common -I platform/common -I sof \
    sof-imx8mp-tac5212-masterlite.m4 > /tmp/test.conf \
    && alsatplg -c /tmp/test.conf -o /tmp/test.tplg \
    && echo "COMPILE_OK, $(stat -c%s /tmp/test.tplg) bytes"
```

STOP si erreur m4 ou alsatplg. Corriger le squelette AVANT deploy.

4. **[V2] T0** sur la board : vérifier nom carte, PCMs, outils.
5. Deploy tplg, reboot.
6. Exécuter T1 (capture dry).
7. Exécuter T2 (playback master avec preset par défaut).
8. Exécuter T3 (master tap reçoit bien le signal).
9. Exécuter T4 (null test bypass aligné).
10. Exécuter T6 (alsactrl live).
11. Exécuter T7 (bytes control mbdrc).
12. Validation utilisateur → commit MVP.

9.3 Séquence 1a.2 — Monitor direct

1. Ajouter `PIPE_2` (SAI RX shared → `eq_iir×8` → buf partagé avec `PIPE_4`).
2. Pattern DAPM cross-pipeline (`PIPELINE_SCHED_COMP_1`) comme dans `kwd`.
3. `critic_analyze` sur le diff.
4. Build + deploy + reboot.
5. Exécuter T8 (latence mic→HP).
6. Validation utilisateur.

9.4 Séquence 1a.3 — Merge mux

1. Remplacer `PIPE_6` direct-playback par `PIPE_4` avec **mux** (2ch master + 6ch monitor).
2. Préparer la lookup table **mux**.
3. `critic_analyze`.
4. Build + deploy + reboot.
5. Exécuter T5 (stress 30 min) + T9 (CPU).
6. Validation utilisateur → Phase 1a OK, passage à Phase 1b.

9.5 Critères de sortie Phase 1a

- T1–T9 tous passent ;
- `test-phase1a.sh` exit 0 ;
- 0 xrun sur un run de 30 min ;
- Latence mic→HP ≤ 4 ms mesurée (T8) ;
- CPU DSP $< 30\%$ (T9) ;
- Documentation à jour : `MASTERING_PLATFORM_PLAN.md` marque Phase 1a comme *done*.

10 Vue courte des phases suivantes

10.1 Phase 1b — Matrice complète

Remplacer le routing fixe Phase 1a par une vraie matrice $N \times M$ via composants **mixer** parallèles (un par bus de sortie) + gains configurables via controls ALSA.

Pré-requis : valider Phase 1a stable + Phase 2 USB opérationnelle (car la matrice a plus de sens quand il y a plus d'inputs).

10.2 Phase 2 — USB gadgets

2 gadgets UAC2 via **configs** sur les deux contrôleurs USB du i.MX8MP :

— USB1 : 8in/8out pour DAW ASIO

— USB2 : 2in/2out pour iOS

Bridges ALSA entre les gadgets et le DSP via **alsaloop** (ou bridge C custom si latence serrée).

10.3 Phase 3 — Effets send (A53 JACK)

Serveur JACK2 avec 4 plugins LV2 (reverb, chorus, flanger, delay). Bridges ALSA $\leftrightarrow$ DSP via 4 PCMs capture + 4 PCMs playback supplémentaires.

Sélection des plugins à faire parmi :

— Reverb : **dragonfly-reverb**, **calf-reverb**

— Chorus : **calf-multichorus**

— Flanger : **calf-flanger**

— Delay : **x42-delay**, **calf-delay**

10.4 Phase 4 — NPU mastering

Training d'un modèle TFLite (from scratch) pour le mastering assisté :

— Input : FFT du master (2048 pts, 20 ms)

— Output : gains EQ 8-10 bandes + seuils mbdrc par bande + amount exciter

— Update rate : 10 Hz

10.5 Phase 5 — Harmonizer + polish

— Harmonizer pitch-shift : plugin LV2 **rubberband** chaîné en A53

— Exciter : plugin LV2 ou module SOF custom (à décider sur CPU budget)

— Flutter UI avancée : presets, scenes, métering en temps réel

11 Annexe — composants SOF mobilisés

11.1 Liste et rôles

Composant	Kconfig	Rôle Phase 1a
dai-copier	natif	Interface SAI7 RX/TX ↔ buffers DSP
host-copier	natif	Interface HOST PCM ↔ buffers DSP
eq_iir	COMP_IIR	3 biquads IIR/ch (fine EQ, flat par défaut)
mux	COMP_MUX	Merge 2ch master + 6ch monitor → 8ch SAI TX
multiband_drc	COMP_MULTIBAND_DRC	Compresseur multibande stéréo (bypass bit-perfect dispo)
volume	natif	Digital Volume Control master (0 dB bit-perfect)
drc	COMP_DRC	Final limiter (bypass bit-perfect dispo)

11.2 Bypass bit-perfect — preuves code

Composant	Fichier	Fonction bypass
multiband_drc	multiband_drc_generic.c:14	multiband_drc_default_pass (≡ audio_stream_copy)
drc	drc_generic.c:473	drc_default_pass (≡ audio_stream_copy)
volume	volume.c	gain Q8.16 = 0x10000 = 1,0 (IPC3) ⇒ bascule is_passthru

11.3 ALSA controls exposés Phase 1a

Nom	Type	Plage / effet
SAI EQ Ch<N> Coeffs	bytes blob	3 biquads IIR (coefficients normalisés)
Master Volume	integer TLV dB	$-\infty..+6$ dB, 0.5 dB step
Master MBDRM Config	bytes blob	Structure sof_multiband_drc_config
Master MBDRM Bypass	switch 0/1	Active multiband_drc_default_pass
Master Limiter Config	bytes blob	Structure sof_drc_config
Master Limiter Bypass	switch 0/1	Active drc_default_pass

11.4 Références datasheet TAC5212

Voir tac5212.pdf dans la racine projet. Sections clés :

- §7.3.9.1 ADC signal chain (biquads, AGC, HPF, gain cal)
- §7.3.9.2 DAC signal chain (biquads, DRC DAC, volume, interpolation)
- §8.2 Programmable coefficient registers

*Document de référence Phase 1a. Tout écart à cette spec doit être tracé dans MASTERING_PLATFORM_PLAN.md et re-soumis à **critic_analyze**.*